TCS Interview Question

Generative AI:

“Generative AI is a type of AI that can generate new content like text or images by learning patterns from large datasets. For example, ChatGPT generates human-like responses by predicting the next word in a sequence.”

- text (ChatGPT)
- images
- code
- audio

Instead of just analyzing data, it generates something new from patterns it learned.

👉 Normal AI = *analyze / classify*
👉 Generative AI = *create*

Examples

- Chatbots → ChatGPT
- Image tools → DALL·E, Midjourney
- Code → GitHub Copilot

How it works

Don’t overcomplicate this.

1. Trained on **huge data (text, images, etc.)**
2. Learns **patterns**
3. Predicts **next word/token**
4. Generates output step-by-step

👉 Example:
If I say: "I am going to the..."

Model predicts → *store / gym / park*

Keyword you MUST know:

👉 **Token**

A token = small unit of text
Example:

```
"I love AI"
→ ["I", "love", "AI"]
```

Agentic AI (very important + trending)

“Agentic AI refers to AI systems that can act autonomously to achieve a goal. Unlike generative AI which only produces outputs, agentic AI can plan, make decisions, and take actions in multiple steps.”
👉 it can **plan** → **decide** → **take actions** → **achieve a goal**

Easy way to understand

🟢 **Generative AI:** "Tell me how to book a ticket"

🔵 **Agentic AI:** "Book the ticket for me"

👉 It will:

1. Search flights
2. Compare prices
3. Select best option
4. (Maybe) proceed with booking

Core idea: Agentic AI behaves like an **agent (assistant with goals)**

It has:

- Goal
- Memory (sometimes)
- Ability to take actions
- Decision-making capability

Flow (VERY useful to remember)

👉 **Goal → Plan → Act → Observe → Repeat**

Real-life examples

- AI assistants that complete tasks end-to-end
- AutoGPT / AI agents
- Smart automation bots
- Future: AI doing your work (emails, scheduling, etc.)

Keywords to drop in interview

- autonomy
- decision making
- goal-oriented
- multi-step reasoning

What is BERT?

👉 **BERT (Bidirectional Encoder Representations from Transformers)** is a language model developed by Google used to understand the meaning of text.

Simple definition (say this in interview)

"BERT is a pre-trained NLP model that understands the context of words in a sentence by looking at both left and right sides of the word."

👉 Bidirectional understanding

Older models:

- Read → left → right
  OR → right → left

👉 BERT:

- Reads → **both directions at the same time**

Example (important)

Sentence: `"I went to the bank to deposit money"`

👉 “bank” = financial meaning

Now: `"I sat near the river bank"`

👉 “bank” = river side

👉 BERT understands this because it looks at **full context**

How BERT works (keep it simple)

Don’t go into heavy math.

Step 1: Tokenization

Sentence → split into tokens

Step 2: Embedding

Words → converted into vectors (numbers)

Step 3: Transformer Encoder

👉 This is the core of BERT
👉 It learns relationships between words

Keyword you MUST know:

👉 **Transformer**

You don’t need deep theory, just say:

“It helps the model understand relationships between words in a sentence.”

Special training techniques (INTERVIEW GOLD)

1. Masked Language Model (MLM)

👉 BERT hides some words and tries to predict them

Example: `"I love [MASK] programming"`

👉 Model predicts → "python"

2. Next Sentence Prediction (NSP)

👉 Checks if one sentence logically follows another

Applications

- Chatbots
- Search engines (Google search improved using BERT)
- Sentiment analysis
- Question answering

Common follow-ups (you must be ready)

Why is BERT better than older models?

“Because it understands context using bidirectional processing, unlike older models which process text in one direction.”

What is the limitation of BERT?

- It is **slow and heavy**
- Requires **high computation**
- Not ideal for real-time low-resource systems

Is BERT generative?

❌ No
👉 It is mainly for **understanding (not generating)**

Mini summary (revise this)

👉 BERT = understands language
👉 Bidirectional
👉 Uses Transformer
👉 MLM + NSP training

Tokenization (core concept)

👉 **Tokenization = breaking text into smaller units called tokens**

“Tokenization is the process of breaking text into smaller units called tokens, which can be words, characters, or subwords. These tokens are then converted into numerical form so that AI models can process them.”

Example

```
"I love AI"
```

👉 Tokens:

```
["I", "love", "AI"]
```

Tokens are **NOT always words**

Types of Tokenization

1. Word Tokenization

```
"I love AI"
→ ["I", "love", "AI"]
```

2. Character Tokenization

```
"I love"
→ ["I", " ", "l", "o", "v", "e"]
```

3. Subword Tokenization (VERY IMPORTANT)

```
"unhappiness"
→ ["un", "happi", "ness"]
```

👉 Used in:

- BERT
- ChatGPT
- modern LLMs

Why do we need tokenization?

👉 Computers don’t understand text directly
👉 They understand **numbers**

So:

1. Text → tokens
2. Tokens → numbers (embeddings)
3. Model processes numbers

RAG (Retrieval-Augmented Generation)

👉 **RAG is a technique where an AI model retrieves relevant information from external data and then generates an answer using it.**

“RAG combines retrieval and generation. It first fetches relevant data from a knowledge source and then uses a language model to generate an accurate response.”

❌ Normal AI (like basic ChatGPT):

- Answers from what it already learned
- Might give outdated or incorrect info

☑ RAG:

1. Searches external data (documents, database, etc.)
2. Gets relevant info
3. Generates answer using that info

Flow (VERY IMPORTANT)

👉 **User Question → Retrieve → Generate → Answer**

Example

User asks:

“What are the hostel rules in my college?”

👉 RAG system:

1. Searches college documents
2. Finds rules
3. Generates answer

👉 Much more accurate than guessing

Components (simple level)

1. **Retriever**
    a. Finds relevant data
2. **Generator (LLM)**
    a. Generates final answer

Keyword you should drop

- “external knowledge”
- “real-time data”
- “reduces hallucination”

👉 That last one is GOLD

Follow-up questions (VERY LIKELY)

Why do we need RAG?

“Because models like ChatGPT are limited to their training data. RAG allows them to access updated and external information, improving accuracy.”

What problem does RAG solve?

- Outdated knowledge
- Hallucinations (fake answers)

Vector Database (important for RAG)

👉 A **vector database stores data in the form of vectors (numerical representations)** so that similar data can be searched quickly.

“A vector database stores embeddings (numerical representations of data) and allows similarity-based search instead of exact matching.”

First understand this (VERY IMPORTANT)

Earlier we said:
👉 Text → tokens → numbers (embeddings)

Example:

```
"I love AI"
→ [0.12, -0.45, 0.88, ...]
```

👉 This list of numbers = **vector**

Why normal database is not enough?

Traditional DB (SQL)

- Works on exact match
- Example:

```
SELECT * FROM users WHERE name = 'John'
```

👉 Needs exact value

Vector DB

- Finds **similar meaning**, not exact match

Example:

- Query: “cheap hostel”

- Stored: “low-cost accommodation”

👉 Still matches ✅

Core idea

👉 **Similarity search (semantic search)**

How it works (simple flow)

1. Text → converted to vector (embedding)
2. Stored in vector DB
3. Query → also converted to vector
4. DB finds **closest vectors**

Real-life example (important)

User asks: “best budget hostel near college”

👉 System:

- Converts query → vector
- Searches similar vectors
- Retrieves relevant data

Connection with RAG (VERY IMPORTANT)

👉 RAG uses:

- **Vector DB for retrieval**
- **LLM for generation**

👉 Full flow: **User → Query → Vector DB → Relevant docs → LLM → Answer**

Keywords to use

- embeddings
- similarity search
- semantic search

Follow-up questions

Why vector DB in RAG?

“Because it allows efficient retrieval of semantically similar data instead of exact matches.”

What is embedding?

“A numerical representation of data that captures its meaning.”

Fine-Tuning (final AI concept)

👉 **Fine-tuning = training a pre-trained model on your own specific data to make it better for your task**

"Fine-tuning is the process of taking a pre-trained model and training it further on domain-specific data to improve performance for a particular task."

Easy way to understand

Imagine:

- Base model = knows general English
- You want = medical chatbot

👉 You fine-tune it with:

- medical data
- hospital conversations

Now it becomes:
👉 **specialized model**

Why we use fine-tuning

- Improve accuracy for specific tasks
- Adapt to domain (finance, medical, etc.)
- Customize behavior

Example

👉 Without fine-tuning:

AI gives generic answers

👉 With fine-tuning:

AI gives **domain-specific answers**

IMPORTANT: Fine-tuning vs RAG

This is **almost guaranteed follow-up**

Difference:

Fine-Tuning	RAG
changes model	does NOT change model
trained on data	fetches data at runtime

static knowledge	dynamic knowledge
expensive	cheaper

Best answer:

“Fine-tuning modifies the model by training it on new data, while RAG retrieves external information dynamically without changing the model.”

When to use what?

Use Fine-tuning when:

- You need **style change**
- Domain is fixed
- Long-term improvement needed

Use RAG when:

- Data changes frequently
- Need real-time info
- Large knowledge base

Keywords to drop

- pre-trained model
- domain-specific
- task-specific improvement

Common follow-ups

Is fine-tuning always required?

“No, many tasks can be handled using prompt engineering or RAG without fine-tuning.”

What is the drawback?

- expensive
- time-consuming
- needs data

Interview-ready answer

“Fine-tuning is the process of training a pre-trained model on specific data to improve its performance for a particular task or domain.”